

Hdoc -Day 24

Name- Aaryan Pratap Singh

Sap ID - 590032842

Batch - 15

1. Algorithm

1. Start.
2. Declare five parameterless void functions:
 - rightTriangle()
 - rowNumberTriangle()
 - naturalNumberPattern()
 - consecutiveTriangle()
 - alphabetPattern()
3. Display the menu:
 - Increasing right-angled triangle
 - Increasing row-number triangle
 - Increasing natural-number pattern
 - Consecutive-number triangle
 - Increasing alphabet pattern
 - Exit
4. Read the user's choice.
5. Use a **switch** statement to call the corresponding function.
6. Inside each function, input and validate the required number of rows.
7. Use nested loops to generate the selected pattern.
8. For the alphabet pattern, accept only $1 \leq n \leq 26$.
9. If the menu choice is invalid, display an error message.
10. Repeat the menu until the user selects Exit.
11. Stop.

The five required patterns and their expected forms are specified in the assignment.

2. Test-Case Table

Test	Choice	Input	Expected result
1	1	4	4-row increasing * triangle
2	2	4	Rows: 1, 2 2, 3 3 3, 4 4 4 4
3	3	4	Rows contain numbers from 1 to row number
4	4	4	Consecutive numbers 1 through 10 in triangle
5	5	4	Rows from A through A B C D

6	1	1 *
7	3	0 Invalid number of rows!
8	5	26 Valid alphabet pattern through Z
9	5	27 Rows must be between 1 and 26!
10	7 —	Invalid menu choice!
11	6 —	Program exits

3. C Program

```
#include <stdio.h>
```

```
void rightTriangle(void);
void rowNumberTriangle(void);
void naturalNumberPattern(void);
void consecutiveTriangle(void);
void alphabetPattern(void);
```

```
int main(void)
{
    int choice;

    do
    {
        printf("\n===== INCREASING PATTERNS
=====\\n");
        printf("1. Increasing right-angled triangle\\n");
        printf("2. Increasing row-number triangle\\n");
        printf("3. Increasing natural-number pattern\\n");
        printf("4. Consecutive-number triangle\\n");
        printf("5. Increasing alphabet pattern\\n");
        printf("6. Exit\\n");

        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice)
        {
            case 1:
                rightTriangle();
                break;
```

```

        case 2:
            rowNumberTriangle();
            break;

        case 3:
            naturalNumberPattern();
            break;

        case 4:
            consecutiveTriangle();
            break;

        case 5:
            alphabetPattern();
            break;

        case 6:
            printf("Exiting program...\n");
            break;

        default:
            printf("Invalid menu choice!\n");
    }

    } while (choice != 6);

    return 0;
}

/* 1. Increasing right-angled triangle */
void rightTriangle(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows!\n");
        return;
    }

    for (i = 1; i <= n; i++)

```

```

    {
        for (j = 1; j <= i; j++)
        {
            printf("* ");
        }
        printf("\n");
    }
}

/* 2. Increasing row-number triangle */
void rowNumberTriangle(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows!\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", i);
        }
        printf("\n");
    }
}

```

```

/* 3. Increasing natural-number pattern */
void naturalNumberPattern(void)
{
    int n, i, j;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {

```

```

        printf("Invalid number of rows!\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", j);
        }
        printf("\n");
    }
}

```

```

/* 4. Consecutive-number triangle */
void consecutiveTriangle(void)
{
    int n, i, j;
    int number = 1;

    printf("Enter number of rows: ");
    scanf("%d", &n);

    if (n <= 0)
    {
        printf("Invalid number of rows!\n");
        return;
    }

    for (i = 1; i <= n; i++)
    {
        for (j = 1; j <= i; j++)
        {
            printf("%d ", number);
            number++;
        }
        printf("\n");
    }
}

```

```

/* 5. Increasing alphabet pattern */
void alphabetPattern(void)
{

```

```

int n, i, j;

printf("Enter number of rows (1-26): ");
scanf("%d", &n);

if (n < 1 || n > 26)
{
    printf("Rows must be between 1 and 26!\n");
    return;
}

for (i = 1; i <= n; i++)
{
    for (j = 1; j <= i; j++)
    {
        printf("%c ", 'A' + j - 1);
    }
    printf("\n");
}
}

```

4. Expected Output for n = 4

Choice 1

```

[aaryanrajput@aaryans-macbook ~ % nano day24.c
[aaryanrajput@aaryans-macbook ~ % clang day24.c -o day24.c
[aaryanrajput@aaryans-macbook ~ % ./day24.c

===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 1
Enter number of rows: 4
*
* *
* * *
* * * *

```

Choice 2

```
===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 2
Enter number of rows: 4
1
2 2
3 3 3
4 4 4 4
```

Choice 3

```
===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 3
Enter number of rows: 4
1
1 2
1 2 3
1 2 3 4
```

Choice 4

```
===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 4
Enter number of rows: 4
1
2 3
4 5 6
7 8 9 10
```

Choice 5

```
===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 5
Enter number of rows (1-26): 4
A
A B
A B C
A B C D
```

Exiting the program

```
===== INCREASING PATTERNS =====
1. Increasing right-angled triangle
2. Increasing row-number triangle
3. Increasing natural-number pattern
4. Consecutive-number triangle
5. Increasing alphabet pattern
6. Exit
Enter your choice: 6
Exiting program...
aaryanrajput@aaryans-macbook ~ %
```

The program was **compiled and executed successfully**, and the four required patterns produced the expected results.